

HOTEC: DOCUMENTACIÓN INTEGRAL DE INGENIERÍA DE SOFTWARE

Entregable Final | Métodos Emergentes en Ingeniería de Software

ÍNDICE GENERAL

- ---

1. MARCO ESTRATÉGICO DEL PROYECTO
 - 1.1 Visión General del Sistema HOTEC
 - 1.2 Objetivos de Automatización
- **2. ESTRUCTURA OPERATIVA Y RESPONSABILIDADES**
 - 2.1 Definición de Roles Técnicos
- **3. INGENIERÍA DE REQUISITOS**
 - 3.1 Requisitos Funcionales y No Funcionales
 - 3.2 Historias de Usuario Críticas
- **4. ARQUITECTURA TÉCNICA Y DESARROLLO (FRONT-END)**
 - 4.1 Tecnologías y Frameworks
 - 4.2 Implementación de Componentes y Rutas
- **5. LÓGICA DE NEGOCIO Y SERVICIOS (BACK-END)**
 - 5.1 Gestión de API y Controladores
 - 5.2 Seguridad y Autenticación
- **6. MODELADO Y PERSISTENCIA DE DATOS**
 - 6.1 Diccionario de Datos
 - 6.2 Entidades Principales en PostgreSQL
- **7. ASEGURAMIENTO DE LA CALIDAD (QA)**

- **7.1 Estrategia de Pruebas Unitarias e Integración**
- **7.2 Validación de Escenarios de Usuario**

1. MARCO ESTRATÉGICO DEL PROYECTO

1.1 Visión General del Sistema HOTECH

HOTECH surge como una respuesta tecnológica a la necesidad de modernizar las operaciones de recepción en la industria hotelera. El sistema se enfoca en la eficiencia operativa, reduciendo los tiempos de respuesta mediante una interfaz web intuitiva conectada a una infraestructura de datos robusta.

1.2 Objetivos de Automatización

El principal objetivo es la digitalización del ciclo de vida del huésped: desde la reserva y el registro (Check-In) hasta la liberación de la habitación (Check-Out), asegurando que el inventario se mantenga actualizado en tiempo real para evitar errores de sobreventa.

2. ESTRUCTURA OPERATIVA Y RESPONSABILIDADES

2.1 Definición de Roles Técnicos

Para la ejecución del proyecto, se estableció una división de tareas basada en competencias técnicas específicas, siguiendo estándares de la industria:

Rol	Responsabilidad Principal
Líder de Proyecto	Gestión de tiempos, coordinación de sprints y cumplimiento de hitos.
Analista	Definición de requisitos funcionales y matriz de trazabilidad.
Diseñador	Diseño de interfaces (UI/UX) y prototipos de alta fidelidad.
Back-End	Lógica de servidor, arquitectura de API y seguridad de datos.
Programador Front-End	Desarrollo de la interfaz reactiva y consumo de servicios API.
Desarrollo (QA)	Codificación de módulos base y aseguramiento de la calidad.

3. INGENIERÍA DE REQUISITOS

3.1 Requisitos Funcionales (RF)

- **RF-01 Autenticación:** El sistema debe restringir el acceso solo a personal autorizado con roles definidos.
- **RF-02 Gestión de Inventario:** Visualización del estado actual de habitaciones (Libre, Ocupada, Mantenimiento).
- **RF-03 Procesamiento de Check-In/Out:** Registro rápido de huéspedes vinculados a habitaciones específicas.

3.2 Historias de Usuario

Como recepcionista, quiero visualizar el tablero de habitaciones para asignar rápidamente una habitación disponible a un huésped que llega sin reserva.

4. ARQUITECTURA TÉCNICA (FRONT-END)

4.1 Tecnologías y Frameworks

Se utiliza React para la construcción de una Single Page Application (SPA), garantizando una experiencia de usuario fluida sin recargas innecesarias de página.

4.2 Implementación de Componentes

```
// Ejemplo de navegación protegida en el Front-End
import { Routes, Route } from 'react-router-dom';
```

```
<Routes>
  <Route path="/login" element={<Login />} />
  <Route path="/dashboard" element={
    <ProtectedRoute>
      <AdminPanel />
    </ProtectedRoute>
  } />
</Routes>
```

```
</ProtectedRoute>  
} />  
  
</Routes>
```

5. LÓGICA DE NEGOCIO Y SERVICIOS (BACK-END)

5.1 Gestión de API y Controladores

El Back-End actúa como el núcleo del sistema, procesando las solicitudes del Front-End a través de una arquitectura RESTful. Los controladores gestionan la lógica de validación antes de interactuar con la base de datos.

5.2 Seguridad y Autenticación

Se implementa un esquema de seguridad basado en tokens (JWT) y el almacenamiento de contraseñas mediante algoritmos de hashing, cumpliendo con los estándares de "Security by Design".

6. MODELADO Y PERSISTENCIA DE DATOS

6.1 Diccionario de Datos

La base de datos PostgreSQL organiza la información en tablas normalizadas para evitar la redundancia y garantizar la integridad referencial.

Tabla	Atributos Principales	Tipo de Dato
Habitación	num_hab, tipo, precio, estado	INT, VARCHAR, DECIMAL, VARCHAR
Huésped	id_huesped, nombre, email, telefono	UUID, VARCHAR, VARCHAR, VARCHAR
Usuario	username, password, rol_id	VARCHAR, TEXT, INT

7. ASEGURAMIENTO DE LA CALIDAD (QA)

7.1 Estrategia de Pruebas

Se aplican pruebas unitarias sobre los modelos de datos y pruebas de integración para validar que la comunicación entre módulos no presente fallos críticos. La fase de QA asegura que el software sea estable antes de su entrega final.

7.2 Validación de Escenarios

Se realizaron simulaciones de carga de trabajo para verificar que el sistema responda eficientemente cuando se registran múltiples Check-Ins simultáneamente, cumpliendo con los requisitos de rendimiento establecidos.

